

SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO2 AT1 - Capstone Infrastructure Sizing Workbook

Guided calibration workbook for VM sizing, virtualization decisions, snapshot planning and VM-container comparison

Assessment Metadata

Course Code	CSA15
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT1 - Virtualization Scenario Problem-Solving
Submission Type	Individual digital workbook based on the same capstone title used in CO1
Faculty	Dr. C. Rajesh Babu
Employee ID	SSETSCS-415
Submission Mode	Completed workbook and evidence repository link through Google Classroom

How This Workbook Works

- Read each guidance block and immediately fill the related response table.
 - Use the same capstone title selected for CO1.
 - Make reasonable assumptions when exact values are unknown. Good assumptions must be written clearly. •
- The goal is not to guess a perfect industry configuration; the goal is to reason technically and justify the first infrastructure plan.
- The final decisions from this workbook will be used again in CO2 AT2 and CO2 AT3.

Technical Tip

A good infrastructure estimate is transparent. If a number is assumed, write why it is assumed. If a number is calculated, show the formula. If a service is selected, justify why it fits the workload.

Student and Capstone Details

Student Name	Karanam Reddy Vishnu Chaithanya
Register Number	192372057
Class / Slot	Slot B
Capstone Title	SentinelX – Cloud Threat Intelligence Using NLP and Serverless Risk Scoring
Capstone Product Type	Web app
CO1 Architecture Reference	CO1 AT2 concept map and service model matrix; CO1 AT3 presentation deck; GitHub repo CSA15-CO-Assessments_192372057

Activity 1: Bring Forward CO1 Capstone Context

Start with the cloud product idea already used in CO1. The infrastructure plan must support that same application, not a new unrelated example.

Capstone Element	Student Entry
Primary users	SOC analysts, IT security teams, and small organizations without dedicated 24/7 security staff
User roles	Analyst (uploads logs, views alerts, triages risk); Admin (manages users, dashboard settings, and integrations)
Main modules	Log ingestion, NLP-based threat analysis, serverless risk scoring, security dashboard, and alerting
Cloud services identified in CO1	AWS Lambda (serverless risk scoring), MongoDB Atlas (data storage), AWS S3 (log/report storage), AWS CloudWatch (monitoring), Firebase/JWT (authentication)
API/backend need	REST API (Node.js + Express) for log submission, triggering NLP analysis, and serving dashboard data
Database need	MongoDB Atlas stores threat logs, NLP analysis output, and computed risk scores
File upload/storage need	AWS S3 stores uploaded raw log files and generated risk reports
Analytics/AI/Python module	NLP-based threat classification and risk scoring, deployed as a serverless function triggered per incoming log

Activity 2: Workload Classification

Classify the capstone workload before selecting a VM. A form-based SaaS app and an AI analytics app should not receive the same infrastructure plan.

Workload Type	Typical Signals	Starting VM Guidance
Light web / form app	Login, CRUD, simple reports, low file upload	1-2 vCPU, 2-4 GB RAM
API backend	Authentication, REST APIs, moderate concurrent requests	2 vCPU, 4-8 GB RAM
Database-heavy app	Search, many records, frequent writes	2-4 vCPU, 8-16 GB RAM
File-heavy app	Images, PDFs, uploads, downloads	2 vCPU, 4-8 GB RAM + separate object storage
Analytics app	Dashboards, aggregation, batch reports	4 vCPU, 16 GB RAM
AI/Python module	Prediction, NLP, image/text processing	4-8 vCPU, 16-32 GB RAM; GPU optional
High traffic app	Large user base and peak demand	Multiple VMs or autoscaling group
Technical Tip Start small for prototype, but do not hide growth. A valid answer can say: prototype uses one VM, pilot separates database, production uses load balancing and managed services.		
Student Decision: My capstone workload type and reason		API backend + AI/Python module (combined). SentinelX exposes a REST API to receive threat logs and serve the dashboard, but the core load driver is the NLP risk-scoring inference. Starting stage: prototype on a single API VM, with NLP inference offloaded to a serverless function so the API tier stays responsive.

Activity 3: User and Traffic Calibration

Use the following formulas to convert user assumptions into approximate traffic.

Metric	Formula		Example
Concurrent Active Users	Registered Users x Active User % x Peak Factor		$500 \times 10\% \times 2 = 100$
Requests per Minute	Concurrent Active Users x Actions per User per Minute		$100 \times 2 = 200$
Requests per Second	Requests per Minute / 60		$200 / 60 = 3.33 \text{ RPS}$
Daily Requests	Daily Active Users x Avg Actions per Day		$200 \times 30 = 6000$
Input	Value	Reason / Assumption	
Registered users expected	200	Estimated analyst seats across a handful of pilot organizations	
Active user percentage	20%	Only on-shift analysts are actively logged in at a given time	
Peak factor	2	Traffic doubles during an active incident or shift-change window	
Actions per user per minute	3	Upload a log, refresh dashboard, open an alert	
Calculated RPS	$80 \text{ concurrent users} \times 3 \text{ actions/min} = 240 \text{ req/min} \rightarrow 240 / 60 = 4 \text{ RPS}$	Derived from the formulas above	
Daily active users	120	Analysts working across morning/evening shifts	
Daily requests	$120 \times 40 = 4800$	Avg 40 actions per analyst per day (uploads, checks, dashboard views)	

Activity 4: CPU Calibration

CPU sizing depends on request rate and processing complexity. Use the score below as a guided estimate.

CPU Driver	Score
Base web/API application	1
Authentication and role management	+1
Moderate API traffic above 5 RPS	+1
Database-heavy search or reporting	+1 to +2
Background jobs or notifications	+1
Analytics or dashboards	+2
AI/Python inference inside server	+2 to +4
Total CPU Score	Recommended vCPU
1-2	1-2 vCPU

3-4		2 vCPU
5-6		4 vCPU
7-9		4-8 vCPU or separate analytics/AI service
10+		Scale out with multiple VMs or managed compute
CPU Driver	Selected Score	Reason

Base web/API application	1	Core REST API for log intake and dashboard data
Authentication and roles	1	JWT-based auth with Analyst/Admin roles
API traffic	0	Calculated load (~4 RPS) is below the 5 RPS threshold for this driver
Database/search/reporting	1	Dashboard queries recent logs and risk scores from MongoDB Atlas
Background jobs	1	Alert notifications sent when a high-risk log is detected
Analytics/dashboard	2	Risk-score trends and reporting views for analysts
AI/Python module	2	NLP threat classification, offloaded to a serverless function but still counted for its own sizing
Total CPU Score	8	
Recommended vCPU	4-8 vCPU, with the AI/NLP workload separated into its own serverless service	

Activity 5: RAM Calibration

RAM must include operating system, runtime, application process, database/cache and a safety buffer.

RAM Formula	Estimated RAM = OS/runtime + app/API + database/cache + analytics/AI + 25% buffer. If database is managed separately, do not include full database memory in the app VM.	
Component	Guidance	Student Estimate
OS + runtime	1-2 GB	1.5 GB
Web/API service	1-2 GB	2 GB (Node.js + Express)
Small database	2-4 GB	0 GB – MongoDB Atlas is managed separately, not hosted on this VM
Medium database/cache	4-8 GB	Not applicable (managed externally)
Analytics/reporting	4-8 GB extra	4 GB – dashboard aggregation runs on the same instance for the prototype
AI/Python module	4-16 GB extra	2 GB – lightweight pre-processing only; heavy NLP inference runs on serverless, not this VM
25% buffer	Subtotal x 0.25	9.5 GB subtotal x 0.25 ≈ 2.4 GB
Final RAM	Round up to standard size	9.5 + 2.4 = 11.9 GB → round up to 16 GB

Activity 6: Storage Calibration

Storage must cover application files, database records, uploads, logs and backup/growth buffer.

Storage Item	Formula / Guidance	Student Estimate
OS and application	20-40 GB	30 GB
Database	Avg record KB x records/day x days x 1.3 / 1024 MB	0 GB on this VM – MongoDB Atlas manages its own storage tier separately
File uploads	Avg file MB x files/day x days x 1.3	5 GB local buffer only – raw log files and reports are moved to AWS S3, not kept on the VM
Logs	Requests/day x log KB x days / 1024 MB	4800 req/day x 2 KB x 30 days / 1024 $\approx$ 0.28 GB → rounded to 2 GB for local app logs (CloudWatch holds the rest)
Backup/growth buffer	Subtotal x 30%	(30 + 5 + 2) = 37 GB subtotal x 0.30 $\approx$ 11.1 GB
Final storage	Round up to standard disk size	37 + 11.1 = 48.1 GB → round up to 50 GB standard disk
Technical Tip Do not store large uploads permanently inside the VM disk when object storage is available. VM disk is for OS and application. Object storage is better for images, PDFs, media and documents.		

Activity 7: Select the VM Plan

Plan	When It Fits	Student Choice
Single VM prototype	Small demo, simple app, low traffic	
Two-tier design	App VM + managed/separate database	
Three-tier design	Frontend, backend/API and database separated	Selected
Containerized backend	Microservices, portable API, CI/CD	Selected (for backend/API)
Serverless function	Event-based task, notification, small automation	Selected (for NLP risk-scoring)
Managed database/storage	Reliability, backup and scaling required	Selected (MongoDB Atlas + S3)

Final VM Plan: selected plan, vCPU, RAM, storage and reason	Three-tier design: React frontend (static hosting/CDN) + one API VM (4 vCPU, 16 GB RAM, 50 GB SSD) running Node.js/Express + dashboard, with MongoDB Atlas as the managed database tier. NLP risk-scoring runs as an AWS Lambda serverless function rather than on the API VM, so unpredictable analysis bursts auto-scale independently without resizing the main server. This keeps the core API responsive, avoids paying for idle AI compute, and matches the CO1 architecture (API Gateway/REST, MongoDB Atlas, S3, CloudWatch).
--	---

Activity 8: Hypervisor and Virtualization Decision

Approach	Meaning	Best Use
Type 1 hypervisor	Runs directly on physical hardware	Production data center and cloud provider infrastructure
Type 2 hypervisor	Runs above an existing OS	Student lab, local testing, VirtualBox/VMware Workstation
Cloud-managed virtualization	Cloud provider hides hypervisor details	Most public cloud VM deployments
Container runtime	OS-level process isolation	Lightweight APIs, microservices, fast deployment
Context	Selected Approach	Justification
Local prototype/testing	Type 2 hypervisor (VirtualBox)	Lets the team test the API and NLP module locally before any cloud spend
Cloud pilot deployment		Virtualization (AWS EC2) AWS handles the hypervisor layer the team focuses on the app, not infrastructure
Production growth scenario	Cloud-managed virtualization with autoscaling group	Handles traffic spikes during widescale incidents without manual resizing
Module suitable for container	Backend/API service (Docker container)	Node.js/Express is portable, restarts fast, and fits CI/CD pipelines

Activity 9: Snapshot and Clone Strategy

Snapshot protects a point in time. Clone creates a duplicate environment. Both are useful, but they are not the same.

Event	Snapshot or Clone?	Frequency / Timing	Retention / Reason
Before major deployment	Snapshot	Immediately before each release	Keep last 3 snapshots for quick rollback
Before database schema change	Snapshot (Atlas backup)	Right before running a migration	Retain until the change is verified stable in production
Before OS/package update	Snapshot	Before applying security patches	Retain 7 days in case the update breaks a service
Before review/demo	Snapshot	The day before a faculty review	Retain until the review is complete
Create testing environment	Clone	Whenever QA tests a new feature branch	Temporary; deleted once testing finishes
Create team demo copy	Clone	Before a major stakeholder demo	Temporary; deleted after the demo, never used as the only backup

Technical Tip

A snapshot is useful for rollback. A clone is useful for parallel testing. A clone should not be treated as the only backup.

Activity 10: VM vs Container Decision

Criteria	Virtual Machine	Container
----------	-----------------	-----------

Isolation	Strong OS-level isolation	Process-level isolation
Startup	Slower	Faster
Resource use	Higher	Lower
Best for	Full OS, legacy app, database VM	Microservices, APIs, portable services
Portability	Moderate	High
Student use	Clear infrastructure learning	Modern deployment learning
Capstone Module	VM / Container / Managed Service	Reason
Frontend	Managed Service	React build is static; served via S3 + CloudFront rather than a running VM
Backend/API	Container	Node.js/Express packaged in Docker for fast, portable, repeatable deployment
Database	Managed Service	MongoDB Atlas handles replication, backup, and scaling without self-hosting

File storage	Managed Service	AWS S3 object storage for uploaded logs and generated reports
AI/Python module	Container / Serverless	NLP risk-scoring runs as an AWS Lambda function for auto scaling, cost-efficient inference
Analytics/dashboard	Container	Runs alongside or beside the API container; can scale independently under load
Notification/background jobs	Serverless	Alert notifications are event-triggered and lightweight, a good fit for a small function

Activity 11: Final Recommendation

Write the final infrastructure recommendation in 8-12 technical lines. Include VM size, hypervisor/virtualization approach, storage plan, snapshot/clone plan and VM-container decision.

1. SentinelX will launch as a three-tier cloud application backed by a single API VM sized at 4 vCPU and 16 GB RAM with a 50 GB SSD, running the Node.js/Express backend and security dashboard.
2. The React frontend is served as a static build through S3 + CloudFront rather than a dedicated server, since it needs no backend compute of its own.
3. MongoDB Atlas is used as a fully managed database tier, so database memory and storage are deliberately excluded from the API VM's sizing.
4. NLP-based threat classification and risk scoring run as an AWS Lambda serverless function, keeping the unpredictable AI workload separate from the core API so bursts auto-scale without resizing the main server.
5. AWS S3 stores raw uploaded log files and generated risk reports, since object storage is more cost-effective and scalable than VM disk for this kind of file growth.
6. Local development and testing use a Type 2 hypervisor (VirtualBox) on student machines, while the cloud pilot deployment moves to AWS EC2's cloud-managed virtualization.
7. As traffic grows toward production, the plan calls for an autoscaling group behind a load balancer rather than manually resizing a single instance.
8. The backend/API and NLP modules are containerized with Docker so they can be redeployed quickly and consistently across local, pilot, and production environments.

9. Snapshots are taken before every major deployment, schema change, OS update, and faculty review/demo, with short retention windows sized to when a rollback would actually be needed.
10. Clones are reserved for QA feature testing and demo copies, and are explicitly not treated as a substitute for real backups.
11. Overall, the plan starts lean (single API VM + managed services + serverless AI) while keeping a clear, documented path to horizontal scaling as SentinelX moves from prototype to pilot to production.

Carry-Forward to CO2 AT2 and CO2 AT3

Output from CO2 AT1	How It Will Be Used Later
VM sizing values	Used as configuration target in CO2 AT2 practical evidence
Hypervisor/virtualization choice	Used to justify deployment environment
Snapshot/clone plan	Used as backup and testing evidence
VM/container comparison	Used in CO2 AT3 infrastructure design case
Final recommendation	Used as the capstone infrastructure baseline

GitHub Submission Checklist

- Completed workbook file.
- README with capstone title, sizing summary and final recommendation.
- Optional architecture diagram or table exported as image/PDF.
- Repository link submitted in Google Classroom.